

C.L.A.R.A.

Contextual Locally Aware Robust Agent

A Personal Autonomous AI System

Author	Alkama Eqbal
Status	Active Development — Voice Live
Hardware	RTX 3050 Mobile (4GB VRAM) · Intel Core i5
Stack	Python · FastAPI · React · Grok API · SQLite · FAISS
Repository	github.com/Angry-Nutrino/AGENT_ZERO

What is CLARA?

CLARA is a personal autonomous AI system — built around the idea that a capable AI shouldn't need to be invoked for every task. Each component is purpose-built rather than assembled from existing frameworks, because the goal was a system that stays present: maintaining context across sessions, running background processes on its own, and routing tasks by reasoning complexity rather than keyword matching. It doesn't just respond — it operates.

The system runs on a single premise: most AI assistants are reactive. They respond when prompted, forget everything between sessions, and have no awareness of the environment they run in. CLARA was built to address all three of those gaps at once, running entirely on consumer hardware with no cloud infrastructure dependency.

This project grew from a genuine desire to have a personal AI that felt like a presence — one that knew context, remembered decisions, and could act without being hand-held. CLARA is the attempt to build that, from scratch, on a 4GB GPU.

The Core Problem It Solves

- Persistent semantic memory across sessions — retrieval by relevance, not just recency
- A continuous background loop that monitors the environment and acts autonomously
- A reasoning pipeline that routes tasks by complexity, escalates on failure, and parallelises independent sub-tasks

How It Works — Architecture

CLARA's architecture is a fully custom autonomous pipeline. Every input — user message, scheduled task, file system change, environment event — flows through the same unified execution path. There are no bypasses.

The Execution Pipeline

```
ALL INPUTS (user / background / environment / system triggers) ↓
EventQueue → OrchestratorLoop → Interpreter (Grok non-reasoning) ↓ Router
→ FAST / CHAT / DELIBERATE ↓ Response → memorize_episode (background
thread)
```

Three Execution Modes

Mode	Trigger	Latency
FAST	Tool known, high confidence, low uncertainty. Interpreter assigns tool + args. Executes directly. Escalates to DELIBERATE on failure, injecting failure context so the reasoning loop knows what was tried.	~2-4s
CHAT	Conversational, no tool needed. Direct streaming via non-reasoning model. Used for opinions, follow-up questions, greetings.	~1.5-2.5s
DELIBERATE	Multi-step reasoning via a ReAct loop (Thought → Action → Observation → Final Answer). Up to 8 turns. Independent tool calls batched and executed in parallel. Used when planning is required or confidence is low.	~5-30s

The Memory System

CLARA maintains a dual-layer memory architecture that persists across sessions:

- **Episodic log** — every interaction is summarised and embedded (MiniLM, 384-dim). Retrieval combines last 3 recency entries + top 2 cosine similarity matches per query. Autonomous system entries are excluded from retrieval to keep user context clean.
- **Long-term vault** — permanent facts extracted from conversations, deduplicated at 0.85 cosine similarity. Personal attributes, project decisions, stable real-world facts. Transient data is explicitly excluded.
- **Archive context injection** — before every Interpreter call, the RAG knowledge base is queried using the same MiniLM embedding already computed for episodic retrieval. Zero extra model calls. If cosine similarity exceeds 0.35, relevant documentation chunks are silently injected into context.

- **Self-knowledge layer** — a structured memory of CLARA's own failure patterns, recovery methods, and architectural facts discovered through use. Auto-populated when CLARA makes and corrects a mistake.
- **Filesystem map** — a hierarchical path tree built from every successful file tool call, injected into every context so CLARA knows what's been explored without re-discovering it.
- **Response style persistence** — verbosity preferences extracted from conversation, written to the user profile, and injected into every subsequent context until changed.

The Knowledge Base (RAG)

CLARA maintains a FAISS vector index built from indexed source documents: resume, CLAUDE.md (live architecture reference), and ROADMAP.md (full implementation history). The knowledge base is:

- Built at startup automatically — runs in a background thread, non-blocking
- Auto-rebuilt when source files change — EnvironmentWatcher detects edits to CLAUDE.md, ROADMAP.md, or any file in the docs folder and triggers a full rebuild + hot-reload without restarting the server
- Queried passively on every request — archive context is injected when relevant, skipped when not, with no latency overhead on misses

The Autonomous Background Loop

CLARA runs continuously. Even when no user is present, the system executes scheduled and event-driven tasks:

- **Scheduled:** health checks every 2 minutes, memory maintenance every 5 minutes, context warmup (embedding sync verification) every 10 minutes
- **Environment-triggered:** file change events with 5-second per-path debounce, memory growth detection, interaction density tracking
- All autonomous actions are logged to episodic memory with an [AUTONOMOUS] prefix — excluded from user query retrieval to prevent noise

Capabilities

The following capabilities are live, implemented, and in daily use:

Reasoning and Task Execution

- Multi-step task decomposition with autonomous tool selection and parallel execution — independent sub-tasks run simultaneously via `asyncio.gather()`
- Web search, Python REPL computation, file system navigation with intelligent path resolution, shell command execution
- Grok Vision — screenshot and image analysis with automatic detail selection (text/code uses high-resolution tiling, layout uses low-resolution for 3-4× speed improvement) and JPEG compression pipeline reducing payload by 5-10×
- Archive search via FAISS — queries the indexed knowledge base for architecture documentation, project history, and personal context
- Real-time streaming responses with thought transparency in a dedicated neural stream panel

Voice I/O

- Push-to-talk (F4) — Faster-Whisper STT transcribes on key release; interrupt support (press F4 while CLARA is speaking)
- Kokoro ONNX TTS with pipelined sentence-by-sentence playback — first audio starts after ~200ms on CUDA
- Acknowledgment system — short spoken cues ("On it.", "Give me a moment.") fire immediately after routing, before execution begins
- Persistent audio streams (mic + speaker always open) — no device reset latency between turns

Tool Ecosystem

- Tool Registry with MiniLM semantic search — 32 tools embedded at startup; pre-Interpreter injection of top-8 most relevant tool schemas per query
- Desktop Commander integration — 26 MCP tools covering file read/write, directory listing, process execution, search, and system introspection
- Mandatory enumeration injection — queries containing "find", "list", "all files" always receive `list_directory` and `start_search` schemas regardless of cosine rank
- Dynamic tool discovery in DELIBERATE — `tool_search` is injected exclusively into the ReAct loop; CLARA can discover unknown tools mid-task by semantic query

Memory and Context

- Semantic episodic memory with cosine similarity retrieval — finds what's relevant across session history, not just what's recent
- Persistent fact vault with deduplication — information accumulates cleanly without bloating context

- Self-knowledge and filesystem map — auto-populating memory layers that grow from actual use
- Follow-up resolution — the Interpreter is explicitly instructed to treat short or demonstrative queries ("in india", "those versions") as continuations of recent context before defaulting to new topic interpretation

Autonomy and Environment Awareness

- Continuous orchestrator loop with priority-based task dispatch, resource conflict detection, and arbitration
- Resource Ledger — read-modify-write protection and write-exclusivity locks prevent concurrent tasks from corrupting shared files
- Task persistence via SQLite with crash recovery — pending tasks survive server restarts and resume correctly
- Live task board — every task state transition (pending → running → completed/failed) is broadcast via WebSocket to the interface in real time

Interface

- React + Vite dashboard with live task board and animated state transitions
- Neural Stream panel split into active task board (top) and thought stream (bottom)
- Token usage pill — per-query breakdown (prompt / completion / cached tokens) shown after each response
- Animated vitals panel with live CPU%, RAM%, and VRAM% bars
- Voice waveform animation and transcript display — user speech appears as a bubble before CLARA responds

Technical Depth

Why Consumer Hardware Matters

Every architectural decision was made under a hard constraint: 4GB VRAM on an RTX 3050 Mobile. Making something production-grade under this constraint required deliberate tradeoffs:

- MiniLM (90MB) is the only model kept permanently in VRAM — used for episodic embedding, archive relevance scoring, and tool registry semantic search
- All LLM reasoning goes to the Grok API — cloud inference with no local VRAM cost

- Vision is handled by Grok Vision API — better accuracy than a local vision model, zero VRAM overhead
- Image compression pipeline (JPEG 85%, ≤1280px) cuts vision call payloads by 5-10×, saving 2-5 seconds per call
- RAG embeddings and Tool Registry embeddings both run on CPU — VRAM stays free for MiniLM
- Kokoro ONNX TTS runs on CUDA via a manually-fixed GPU session (bypassing a broken find_spec check in the library) — ~200ms first-audio latency

Key Engineering Decisions

Interpreter	Non-reasoning Grok (~950ms) parses every input into structured JSON: intent, tool, args, confidence, uncertainty, requires_planning. Replaced a local MiniLM+Phi-3 gatekeeper — higher accuracy, lower VRAM, no local model dependency.
Parallel tool batching	Independent tool calls in a single DELIBERATE turn execute via asyncio.gather(). Bitcoin price + exchange rate → one pass, not two sequential calls.
Archive injection	Pre-Interpreter FAISS query reuses the MiniLM embedding already computed for episodic retrieval. Zero extra model calls. Sub-10ms. Injects relevant documentation chunks when cosine similarity ≥ 0.35.
RAG auto-rebuild	EnvironmentWatcher detects changes to CLAUDE.md, ROADMAP.md, and docs/. Triggers full FAISS rebuild in background thread + hot-reload of in-memory engine. Knowledge base stays current without restart.
Tool Registry	All 32 tools (6 native + 26 DC) are embedded at startup using MiniLM. Per-request: top-8 most relevant schemas by cosine similarity are injected into Interpreter context as [DISCOVERED_TOOLS]. DC descriptions are stripped of boilerplate before embedding to improve signal quality.
Voice pipeline	Persistent InputStream (mic) and OutputStream (speaker) opened once at startup. No global sd.play/wait calls — those trigger WASAPI device resets on Windows. Pipelined TTS: synthesizer fills an audio queue, playback loop writes 0.2s chunks continuously. First audio after ~200ms on CUDA.
Hallucination detection	The ReAct loop uses "Glint" internally instead of "Observation" — removing the training prior that causes models to generate fake observations. Inline detector catches cases where the model emits a Glint without calling a tool and injects a corrective system message.

Filesystem resolution	For named file lookups, <code>start_search</code> runs first — confirms existence and returns exact path in one call with no chunk-limit risk. <code>list_directory</code> is a fallback only if search returns nothing. Interpreter flags inferred paths (not explicitly given by user) as <code>requires_planning</code> so FAST never executes blind filesystem calls.
Embedding sync	System log entries (health checks, task retries) append zero-vector embeddings to maintain array index parity with the episodic log. A drift in lengths silently disables semantic retrieval — this prevents that.
Task board broadcast	Every task state transition fires a WebSocket event to all connected clients via the orchestrator's injected broadcast callback — no circular imports, no polling.

What's Next

CLARA is not a finished product — it is an evolving system with an explicit, sequenced roadmap.

Immediate: Streaming Migration (Pattern B)

The current ReAct loop makes multiple sequential LLM calls (one per turn). The next architectural shift moves to a single streaming LLM call with mid-stream tool execution — tools fire during the stream, results are injected, and the model continues without a round-trip. This eliminates multi-turn overhead entirely and is the foundation for sub-second tool execution latency.

Near-term: Ambient Awareness

The long-term vision is closer to JARVIS than to a chatbot. Building toward that requires:

- **Proactive insight engine** — a background task that reviews context and decides when something is worth saying without being asked
- **External event integration** — WhatsApp messages, calendar events, and notifications as EventQueue inputs. CLARA receives the event, understands context from memory, and responds with awareness
- **Structured life context** — a typed layer for fitness, nutrition, goals, and habits that CLARA can query and reason over, not just store as free-text vault facts

Long-term

- Episodic log pruning with archival — practical necessity within 2-3 months at current growth rate
- Vault contradiction detection — when a new fact contradicts an existing one, replace rather than append
- Brave Search integration — multi-tier search API with dedicated LLM Context endpoint (returns raw page content for Grok to reason over, not a synthesized answer)

Current State — May 2026

The following has been designed, implemented, tested, and is running in daily use across ~30 source files:

- ✓ Unified pipeline: EventQueue → Orchestrator → Interpreter → Router → FAST/CHAT/DELIBERATE
- ✓ Dual-layer semantic memory with cosine retrieval and vault deduplication
- ✓ Conditional RAG archive injection (FAISS, MiniLM, CLAUDE.md + ROADMAP.md + resume indexed)
- ✓ RAG auto-rebuild on source file changes via EnvironmentWatcher + hot-reload without restart
- ✓ Grok Vision with JPEG compression, resize pipeline, and auto detail selection
- ✓ Voice I/O — Faster-Whisper STT + Kokoro ONNX TTS, push-to-talk, pipelined playback, ~200ms first-audio latency
- ✓ Tool Registry — 32 tools embedded, top-8 pre-Interpreter injection, mandatory enumeration injection
- ✓ Desktop Commander MCP integration — 26 tools via semantic registry
- ✓ ReAct loop with parallel tool batching, 16 explicit behavioral rules, hallucination detection, and loop safety net
- ✓ Filesystem resolution strategy — start_search first, list_directory as fallback, path uncertainty routing
- ✓ Resource Ledger — write-exclusivity locks for concurrent task safety
- ✓ Token usage tracking — per-query breakdown across all LLM calls, Neural Stream pill display
- ✓ Self-knowledge and filesystem map — auto-populating memory layers
- ✓ Live task board with real-time WebSocket state broadcasts from the orchestrator
- ✓ Response style persistence, known file location injection, follow-up resolution in Interpreter
- ✓ EnvironmentWatcher with 5-second per-path debounce
- ✓ Episodic embedding sync with zero-vector system entries and self-repair warmup

- ✓ Persona grounding — Alkama-anchored identity, no fabricated personal history, no hallucinated architecture claims
- ✓ 26 implementation briefs completed, each reviewed and signed off before the next began

What This Demonstrates

Building CLARA required working across async Python systems, LLM prompt design, vector databases, React frontend, WebSocket protocol, SQLite persistence, GPU memory management, audio I/O, and consumer hardware optimisation simultaneously. The system works because all of them work together correctly — not in isolation.

More than technical breadth, the project demonstrates a way of working: reason through decisions fully before implementing, make surgical changes with explicit review gates, and never rush past a problem that will cost more later. Every brief in this project was written and reviewed before a single line of production code was changed.

Alkama Eqbal · Final-year Computer Science, RNSIT · Graduating mid-2026
alkamaeqbal@gmail.com · github.com/Angry-Nutrino · +91-93419-47789